

Nama: Azzam Prastianto

NIM: J0403251153

Kelas: A2

STUDI KASUS

A. Node (Vertex)

Node 0: Router

Node 1: Laptop Lenovo

Node 2: Laptop Acer

Node 3: Laptop Asus

Node 4: Laptop Axioo

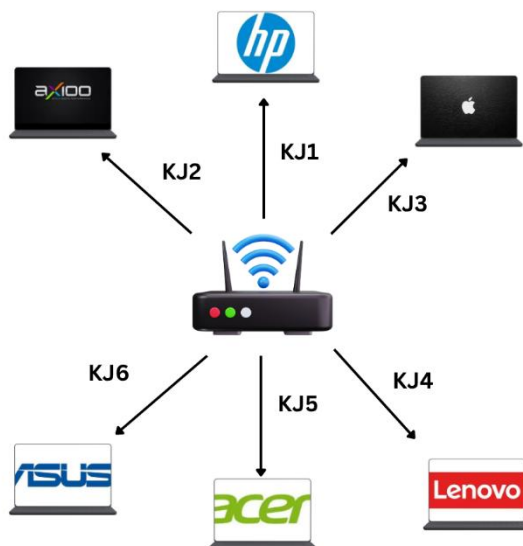
Node 5: Laptop Macbook

Node 6: Laptop HP

B. Edge

- Koneksi Jaringan 1 (KJ1)
- Koneksi Jaringan 2 (KJ2)
- Koneksi Jaringan 3 (KJ3)
- Koneksi Jaringan 4 (KJ4)
- Koneksi Jaringan 5 (KJ5)
- Koneksi Jaringan 6 (KJ6)

C. Gambar Desain Graph



D. Implementasi pada Python

- Adjacency List dan outputnya

```
from collections import defaultdict
graph = defaultdict(list)
graph["Node 0"].append("Node 1")
graph["Node 0"].append("Node 2")
graph["Node 0"].append("Node 3")
graph["Node 0"].append("Node 4")
graph["Node 0"].append("Node 5")
graph["Node 0"].append("Node 6")
graph["Node 1"]
graph["Node 2"]
graph["Node 3"]
graph["Node 4"]
graph["Node 5"]
graph["Node 6"]

print(dict(graph))
```

```
{'Node 0': ['Node 1', 'Node 2', 'Node 3', 'Node 4', 'Node 5', 'Node 6'], 'Node 1': [], 'Node 2': [], 'Node 3': [], 'Node 4': [], 'Node 5': [], 'Node 6': []}
```

- Adjacency Matrix dan outputnya

```
def createGraph(V, edges):
    adj = [[] for _ in range(V)]

    for it in edges:
        u = it[0]
        v = it[1]
        adj[u].append(v)

    return adj

if __name__ == "__main__":
    V = 7

    edges = [[0, 1], [0, 2], [0, 3], [0, 4], [0, 5], [0, 6]]

    adj = createGraph(V, edges)

    print("Adjacency List Representation:")
    for i in range(V):
        print(f"{i}: ", end=" ")
        for j in adj[i]:
            print(j, end=" ")
        print()
```

```
Adjacency List Representation:
0: 1 2 3 4 5 6
1:
2:
3:
4:
5:
6:
```

E. Analisis singkat

1. Jenis Graph (Directed / Undirected)

Berdasarkan rancangan visual dan implementasi kode pada dokumen studi kasus, graf ini dikategorikan sebagai directed graph. Pada desain graf, terdapat tanda panah yang membentang dari Router menuju ke masing-masing laptop, hal ini menunjukkan bahwa relasi jaringan hanya terjadi satu arah. Hal ini juga didukung oleh implementasi kode Python yang disertakan, di mana data relasi hanya ditambahkan ke keys dengan nilai "Node 0", seperti pada perintah `graph["Node 0"].append("Node 1")` hingga menuju Node 6, sementara keys node 1 sampai 6 menghasilkan list kosong (`[]`) karena kode tidak mendefinisikan adanya sisi (edge) balik yang mengarah kembali ke Node 0.

2. Alasan Memilih Edge

Kita menggunakan edge yang dinamakan KJ1 sampai KJ6 untuk menunjukkan jalur koneksi dari penyedia jaringan ke penerima. Di sini, edge bertugas menghubungkan Node 0 (Router) sebagai pusat jaringan ke enam laptop yang berbeda (Node 1 sampai 6). Adanya edge ini sangat penting. Kalau garis penghubung ini tidak ada, router dan laptop-laptop itu cuma bakal jadi perangkat yang berdiri sendiri dan tidak akan bisa membentuk satu kesatuan jaringan berbentuk bintang (star topology).

3. Representasi Mana yang Lebih Cocok

Adjacency List adalah pilihan yang paling pas dan efisien untuk jaringan model terpusat seperti ini. Alasannya, Adjacency List hanya mencatat perangkat yang benar-benar saling terhubung. Bisa dilihat di output kodenya, cuma Node 0 yang punya daftar koneksi ke Node 1 sampai 6, sedangkan node laptop lainnya dibiarkan kosong sehingga tidak makan tempat. Sebaliknya, kalau kita memaksakan pakai Adjacency Matrix ukuran 7x7, sistem harus menyiapkan 49 kotak data. Padahal sebagian besar kotak itu cuma bakal diisi angka nol karena laptop-laptopnya tidak saling terhubung langsung. Jadi, Adjacency List jelas jauh lebih ringan buat memori program.

4. Klasifikasi Graph (Dense atau Sparse)

Jaringan ini masuk ke dalam kategori graf jarang (sparse graph). Kenapa bisa dibilang jarang? Karena jumlah koneksi aslinya sangat sedikit kalau dibandingkan dengan total koneksi maksimal yang sebenarnya bisa dibuat oleh ketujuh node tersebut. Di rancangan ini cuma ada tepat 6 garis edge yang menyambungkan router ke tiap laptop. Beda ceritanya kalau semua laptop juga diatur buat saling terhubung dan berbagi data satu sama lain secara langsung

(peer-to-peer), pasti koneksinya akan jauh lebih padat. Nah, karena garis penghubungnya sedikit dan tidak saling menyilang antar laptop, makanya graf ini disebut graf jarang.

Kesimpulan

Berdasarkan analisis studi kasus jaringan yang telah dijabarkan, dapat disimpulkan bahwa rancangan jaringan ini membentuk sebuah topologi bintang (star topology) yang menghubungkan satu Router utama (Node 0) ke laptop (Node 1 hingga 6) melalui jalur koneksi KJ1-KJ6. Jaringan ini secara tegas dikategorikan sebagai directed graph karena aliran relasinya hanya dirancang berjalan satu arah dari router menuju laptop. Dalam hal efisiensi struktur data, penerapan Adjacency List terbukti menjadi opsi representasi yang paling tepat dan ringan untuk memori komputasi dibandingkan Adjacency Matrix, karena metode ini secara spesifik hanya mencatat koneksi jaringan yang benar-benar aktif. Terakhir, minimnya jumlah garis penghubung yang terbentuk (hanya 6 edge) dibandingkan batas koneksi maksimal yang ada, serta ketiadaan koneksi silang antar laptop, menempatkan graf jaringan terpusat ini ke dalam klasifikasi sparse graph.